
Loop-R1: Learning Semantic Reasoning for Loop Invariant Synthesis in Large Language Models

Jian Jornbowl Wang¹ Lili Quan¹ Qiang Hu² Mingfei Cheng¹

Yi Li³ Xiaofei Xie¹

¹Singapore Management University ²Tianjin University ³Nanyang Technological University

Abstract

Loop invariant generation is a fundamental yet challenging problem in formal program reasoning. Recent work has shown that large language models (LLMs) can improve invariant inference through prompting and iterative refinement. However, these approaches primarily rely on powerful pretrained models (e.g., GPT-4-class models) and largely treat LLMs as static generators, without investigating how invariant reasoning itself can be systematically enhanced through learning. In this paper, we present the first systematic study of fine-tuning LLMs to improve their semantic reasoning capabilities for loop invariant inference. We introduce two primary contributions: (1) a large-scale, reasoning-oriented dataset comprising C/C++ programs, formally verified invariants, and step-by-step reasoning traces; and (2) a unified training pipeline that integrates supervised fine-tuning (SFT) and reinforcement learning from formal feedback (RL). Our evaluation across multiple open-source, instruction-tuned LLMs demonstrates that reasoning-aware SFT, particularly when combined with RL, significantly improves semantic correctness and generalization, while RL alone is difficult to train and yields limited gains in this domain.

1 Introduction

Loop invariants are a cornerstone of formal reasoning about programs, underpinning a wide range of software engineering tasks such as program verification, automated repair, and termination analysis. A loop invariant is a logical property that must hold before and after each iteration of a loop, forming the foundation of deductive reasoning about loop programs. By summarizing the effect of unbounded iterations, invariants play a central role in verification frameworks grounded in Hoare logic [14, 26] and its extensions [46, 42, 25, 7, 6]. Despite decades of research on automated invariant generation, producing semantically correct and sufficiently strong invariants remains a fundamental challenge, particularly for programs involving complex control flow and arithmetic relationships.

Extensive research has investigated the automated generation of loop invariants, spanning a spectrum from formal static analysis [20, 43] to modern data-driven approaches [42, 47]. Static methods primarily rely on abstract interpretation, which iteratively computes fixed points over predefined domains, and template-based synthesis, which transforms invariant discovery into a constraint-solving problem. Conversely, dynamic and learning-based approaches decouple the search process from the program logic by inferring patterns from execution data. These methods often utilize Counterexample-Guided Inductive Synthesis (CEGIS) to iteratively refine candidates, represented by decision trees [18, 21], reinforcement learning agents [47], or continuous logic networks [47, 58], using feedback from formal verifiers. Despite this progress, existing methods are often constrained by fixed invariant templates or the scalability of learning components when faced with complex control flows, non-linear arithmetic, and heap-manipulating behaviors.

Recent advances in large language models (LLMs) have sparked growing interest in leveraging pretrained models for loop invariant generation. Existing approaches typically frame this task as a zero-shot or prompt-based problem, relying primarily on the model’s pretrained knowledge. Because LLMs rarely produce semantically correct invariants in a single attempt, prior work commonly adopts ranking-based or iterative refinement strategies to improve candidate quality. Ranking-based methods [10] generate multiple invariant candidates and select those most likely to satisfy formal constraints, while iterative refinement approaches [56, 9] incorporate feedback from formal verification to repeatedly prompt the model toward improved solutions.

Despite recent advances, current LLM-based methods primarily rely on prompt engineering to exploit the capabilities of pre-trained models, failing to explicitly enhance the models’ underlying reasoning architecture. However, loop invariant generation is not merely a problem of linguistic or syntactic pattern matching; it is a form of semantic program reasoning that requires internalizing the inductive structure of program behavior. When the reasoning capacity of a model, particularly resource-constrained or general-purpose LLMs, is insufficient, the synthesis process often devolves into a prolonged iterative refinement cycle. This results in a high volume of model queries and substantial computational overhead. In the worst case, this process functions as a stochastic guess-and-check loop rather than a principled reasoning. Consequently, improving the intrinsic reasoning capabilities of LLMs is essential to bolster both effectiveness (*ensuring semantic correctness and invariant strength*) and efficiency (*minimizing the number of refinement iterations*).

Motivated by these limitations, we focus on enhancing the intrinsic reasoning capabilities of LLMs for loop invariant generation. Recent advances in supervised fine-tuning (SFT) and reinforcement learning (RL) have substantially improved LLM performance on challenging reasoning tasks such as mathematical problem solving and program synthesis. However, how these learning paradigms translate to semantic program reasoning, and in particular to loop invariant generation, remains largely unexplored. A central challenge lies in the lack of reasoning-grounded training data (with explicit reasoning steps) and a systematic training pipeline tailored to the formal requirements of invariants. In this work, we address this gap by constructing a semantically validated dataset aligned with invariant proof obligations and by developing a fine-tuning framework that integrates both supervised and reinforcement learning. We utilize this framework to systematically evaluate the impact of these learning paradigms on the correctness, generalization, and efficiency of the synthesized invariants.

Specifically, for dataset construction, we first prompt a pretrained code LLM to synthesize candidate programs following prior work [55], apply Tree-sitter-based syntactic filtering to retain only those with valid loops, infer candidate invariants using state-of-the-art inference tools, and formally verify their correctness via Satisfiability Modulo Theories (SMT) solvers. For each verified program-invariant pair, we further prompt multiple strong LLMs to generate reasoning traces and select high-quality ones as ground-truth supervision through three LLM-based evaluation strategies, yielding a training dataset of 28,420 (*program, invariant, reasoning*) triples. On top of this dataset, we adopt three learning strategies. SFT trains on the full reasoning-annotated dataset with explicit step-by-step supervision. RL trains on programs without reasoning traces, allowing the model to refine its behavior through self-exploration guided by verification feedback rather than mimicking predefined trace styles. SFT+RL first initializes the model with reasoning-aware supervision and then refines it through exploration-driven reinforcement learning.

We evaluate our approach on three representative open-source, instruction-tuned LLMs (Qwen2.5-Coder-1.5B, Qwen3-1.7B, and Llama-3.2-1B), fine-tuning each under the SFT, RL, and SFT+RL settings. Reasoning-aware fine-tuning with SFT and SFT+RL consistently improves invariant inference across both benchmarks, with the combined SFT+RL achieving the most robust gains and, in some cases, enabling compact open-source models to rival or surpass larger commercial instruction-tuned LLMs. In contrast, RL alone yields limited or unstable gains. We further analyze these results to derive insights for designing effective training signals for semantic program reasoning.

In summary, we make the following main contributions: 1)**Reasoning-Oriented Dataset.** We construct a high-quality, large-scale dataset consisting of 28,420 C/C++ programs with explicit loop constructs, verified loop invariants, and curated step-by-step reasoning traces. 2)**Fine-Tuning Pipeline.** We develop a unified training pipeline that supports SFT and RL, enabling controlled analysis of how different learning signals shape semantic program reasoning. 3)**Systematic Empirical Study.** We conduct the first systematic study of explicit reasoning enhancement for loop invariant inference, showing that combined SFT + RL training yields significant gains.

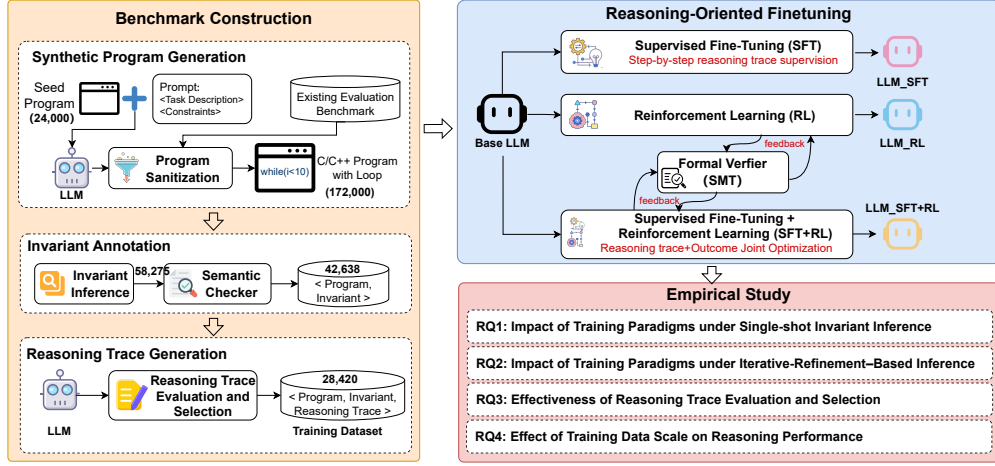


Figure 1: Overview of the whole framework.

2 Methodology

2.1 Preliminary

A loop invariant is a logical property that must hold before and after each iteration of a loop. By standard Hoare logic [26], for a loop of the form `while B do S`, loop invariant inference aims to identify an invariant I such that:

$$\frac{P \Rightarrow I \quad \{I \wedge B\} S \{I\} \quad (I \wedge \neg B) \Rightarrow Q}{\{P\} \text{ while } B \text{ do } S \{Q\}} \quad (1)$$

where P is the pre-condition, Q is the post-condition, and S is the loop body with B as the loop condition. Essentially, Equation (1) includes the following three requirements: 1) *Pre-condition Satisfaction*: the invariant must hold before entering the loop ($P \Rightarrow I$); 2) *Invariant Preservation*: if the invariant holds at the beginning of an iteration and the loop guard is true, it must remain true after executing the loop body ($\{I \wedge B\} S \{I\}$); 3) *Post-condition Satisfaction*: upon loop termination, the invariant together with the negation of the guard must imply the postcondition ($(I \wedge \neg B) \Rightarrow Q$).

In practice, SMT solvers (e.g., Z3 [13]) are commonly used to check these conditions. When a condition fails, the solver returns a concrete counterexample, which can be leveraged to refine the current invariant within an iterative invariant inference framework.

2.2 Overview

Figure 1 overviews our framework, which consists of three components: dataset construction, reasoning-oriented fine-tuning, and empirical study. For dataset construction, we first generate C programs with loops by prompting a pre-trained code LLM with seed snippets from PYX [16], followed by lightweight syntactic filtering. We then annotate loop invariants using state-of-the-art inference tools with SMT-based semantic checking. Finally, we generate reasoning traces with multiple strong LLMs and select high-quality ones as ground-truth supervision. We then fine-tune LLMs under three training strategies (SFT, RL, and SFT+RL) and evaluate them under both single-shot and iterative-refinement inference settings, with additional analysis on the impact of reasoning-trace selection and training dataset size.

2.3 Benchmark Construction

2.3.1 Synthetic Program Generation.

Programs in a loop invariant dataset must contain explicit loop constructs and well-defined invariants [54]. Collecting such programs directly from real-world codebases is challenging due to the lack of invariant annotations, uncontrolled complexity, and interactions with external resources. We therefore adopt a synthetic data generation strategy to control both program complexity and structural diversity.

A key challenge in synthesizing programs with LLMs is ensuring diversity, as naive prompting tends to produce homogeneous loops dominated by simple arithmetic patterns. Inspired by prior work [16], we adopt a few-shot prompting strategy that conditions generation on a curated set of structurally diverse seed programs from the PYX dataset [16]. For each seed snippet, we prompt multiple LLMs to generate 67 candidate C programs. Each prompt consists of three components (see Figure 2 in Appendix B.1 for details): (1) a task instruction, (2) a generation specification, and (3) a seed code snippet. The specification defines concrete requirements for code generation, including the presence of a main function, referring to the structure of the seed loop, the use of nondeterministic initialization for variables, and the application of assume statements to constrain variable ranges. It also explicitly enforces diversity constraints, such as varying loop difficulty (easy, medium, hard) and functionality (e.g., pattern matching, numerical computation, array traversal, and index tracking).

Starting from 24K seed programs from PYX, this process yields approximately 1.53M candidate C/C++ programs. We then perform a multi-stage quality and sanitization pipeline. Specifically, we use Tree-sitter [4] to retain only programs containing valid loop constructs, eliminate near-duplicate programs via structural similarity clustering, and remove any samples exhibiting significant n -gram overlap with our test benchmarks (Section 3.1) to prevent data leakage. After this filtering process, we retain 172K high-quality programs for subsequent invariant annotation and training.

2.3.2 Invariant Annotation.

Manual annotation of loop invariants is impractical at scale due to the inherent difficulty of invariant inference and the deep program understanding required. We therefore employ two state-of-the-art LLM-assisted invariant inference systems, LaM4Inv [56] and Clause2Inv [9], both following a guess-and-check paradigm in which candidate invariants are proposed by LLMs and validated by SMT solvers or bounded model checking (BMC). We run LaM4Inv as the default engine and fall back to Clause2Inv when it fails. Programs for which neither tool yields a verified invariant are discarded. This pipeline produces verified invariants for 58,275 programs.

However, formal verification alone does not guarantee semantic correctness. We observe that existing inference frameworks rely on translation pipelines that encode C-level invariants into SMT formulas, and translation bugs can introduce semantic drift such that an invariant validated by the SMT solver no longer matches the original C-level program. An illustrative example of such translation-induced false positives is provided in Appendix B.2.

To mitigate this issue, we develop a semantic consistency filter. For each inferred C-level invariant, we translate it into an SMT formula, map the SMT formula back into a C expression, and check whether the reconstructed invariant is semantically equivalent to the original. Only invariants passing this check are retained as ground-truth labels, leaving 42,638 program-invariant pairs after filtering.

2.3.3 Reasoning Trace Generation

To provide explicit supervision for invariant derivation, we augment each verified program-invariant pair with a reasoning trace reconstructing the inductive discovery process. Following prior work [61], we leverage strong LLMs to generate candidate traces, prompting them to structure each trace around four components: *Loop and State Abstraction*, *Behavioral Pattern Extraction*, *Constraint Generalization*, and *Invariant Synthesis*. Detailed component definitions are provided in Appendix B.3. Although directly inducing correct invariants from code alone remains difficult, providing the ground-truth invariant together with these derivation principles enables the model to produce semantically grounded traces that reflect plausible discovery processes. To obtain a diverse pool, we query 7 LLMs multiple times to generate 19 candidate traces per pair.

Due to hallucinations and output variability, candidate traces may contain logical errors, be overly verbose, deviate from program semantics, or omit key inductive steps. Following prior work on chain-of-thought faithfulness [30, 33, 60], we employ LLM-based evaluators to score each candidate trace under three explicit quality criteria. *Logical Consistency* (q_{lc}) measures whether the trace and target invariant are mutually consistent under program semantics, with each step grounded in concrete program behavior. *Structural Clarity* (q_{sc}) measures how coherently the trace progresses through the four derivation steps, avoiding redundant or tangential content. *Training Influence* (q_{ti}) measures the trace’s expected training utility, favoring traces associated with rare or structurally complex invariants. Full criterion specifications are provided in Appendix B.4. Each trace is assigned a quality vector

$\mathbf{q} = (q_{lc}, q_{sc}, q_{ti}) \in [1, 5]^3$ by the LLM-based judge, where each criterion is scored from 1 (worst) to 5 (best).

2.4 Reasoning-oriented Fine-tuning

Our pipeline integrates supervised fine-tuning (SFT) on reasoning traces with reinforcement learning from formal verification feedback, enabling controlled analysis of trace-level supervision and outcome-level semantic rewards.

Supervised Fine-Tuning (SFT) We fine-tune the LLM on curated triples (P, T, I) via standard cross-entropy loss. Each training instance pairs the program P and a step-by-step derivation instruction as input with an output containing the reasoning trace T enclosed in `<think>...</think>` tags followed by the invariant I in `<answer>...</answer>` tags. The complete prompt template is provided in Appendix B.5.

Reinforcement Learning with Formal Feedback We adopt Group Relative Policy Optimization (GRPO) [22] but replace standard human-preference feedback [40] with *formal verification feedback*, yielding a fully automated and objective training signal. During training, the LLM is given a program P and generates an invariant (optionally with a reasoning trace), evaluated by a composite reward function:

$$R(\text{output}; P) = w_1 R_{\text{smt}} + w_2 R_{\text{bmc}} + w_3 R_{\text{stab}} + w_4 R_{\text{fmt}}, \quad (2)$$

where R_{smt} rewards invariants satisfying inductiveness checks in an SMT solver (e.g., Z3); R_{bmc} rewards invariants passing bounded model checking via ESBMC [39] with bounded unwinding $k < 10$ following prior work [57, 56]; R_{stab} rewards invariants passing the semantic consistency check from Section 2.3.2; and R_{fmt} rewards outputs in a clean, easy-to-parse invariant format. Each component is binary (1 if satisfied, 0 otherwise), with weights w_i balancing relative importance. To support efficient optimization, we parallelize solver calls, cache previously evaluated invariants, and inject minor format and prompt variations to maintain generation diversity.

Combined Supervised and Reinforcement Learning (SFT+RL) SFT and RL provide complementary signals: SFT internalizes abstraction patterns through trace-level supervision, while RL refines the model through outcome-driven exploration guided by formal verification. We combine them by first applying SFT on the curated dataset to establish a reasoning-aware initialization, then continuing with RL for further refinement. This hybrid approach balances *trace-level supervision* with *semantic, output-driven optimization*, yielding models that are both more robust and more generalizable.

3 Experimental Setup

3.1 Dataset

Training Dataset. We prepare separate training datasets for SFT and RL. From the full set of 42,638 program-invariant pairs, we select samples with high-quality reasoning traces by retaining only those whose scores on all evaluation criteria are at least 4, resulting in 28,420 triples for supervised fine-tuning. For reinforcement learning, we construct a 5,000-instance RL training set. Specifically, 4,000 instances are sampled uniformly at random from data not used for SFT training, while the remaining 1,000 instances form a hard-case subset collected from earlier guess-and-check failures (i.e., cases that could not be successfully resolved during the annotation stage described in §2.3.2). This design increases the prevalence of challenging cases during RL training. We deliberately limit the size of the RL training set due to the substantially higher computational cost of reinforcement learning, which requires repeated model invocations and formal verification for reward computation.

Evaluation Benchmarks. We evaluate on two widely adopted benchmarks: **Clause2Inv-Bench** [9] (366 linear and non-linear invariant synthesis tasks) and **InvBench** [53] (226 SV-COMP verification tasks), yielding 592 tasks in total. Details are provided in Appendix C.2.

3.2 Evaluation Protocol and Models

We evaluate model performance under two representative LLM-based invariant inference paradigms.

Single-shot invariant inference directly prompts the model to generate one or more loop invariants without incorporating formal verification feedback during inference. In this setting, we follow the benchmark-specific evaluation protocols: the formal verifier Z3 is used for Clause2Inv-Bench, and UAutomizer is used for InvBench.

Iterative-refinement-based invariant inference follows existing guess-and-check refinement frameworks that iteratively generate intermediate artifacts or refined invariant candidates while incorporating formal verification feedback. We use LaM4Inv [56] as a representative tool for this paradigm and adopt the default formal verifier provided by its implementation.

Reasoning-aware fine-tuning is performed on three small open-source instruction-tuned models due to the need of huge resource, i.e., *Qwen2.5-Coder-1.5B-Instruct*, *Qwen3-1.7B-Instruct*, and *Llama3.2-1B-Instruct*. For each model, we produce variants trained under different learning paradigms, including SFT, RL, and SFT+RL.

For evaluation, except for the fine-tuned models, we include larger commercial reasoning LLMs, commercial instruction-following LLMs, and the original instruction-tuned versions of the same open-source models without reasoning-aware fine-tuning. The commercial reasoning models include *GPT-5-mini*, *GPT-o4-mini*, *DeepSeek-reasoner*. Commercial instruction-following models include *GPT-4o-mini*, and *DeepSeek-chat*. The open-source models include *Qwen2.5-Coder-7B-Instruct*, *Qwen3-8B-Instruct*, and *Llama3-8B-Instruct*.

Evaluation Metrics Following prior work on LLM-based program synthesis [36], we evaluate single-pass invariant inference using the unbiased **pass@k** metric [11], which estimates the probability that at least one of k sampled candidates passes formal verification. The detailed formulation is provided in Appendix C.1. For each LLM, we report **pass@1*** under greedy decoding (temperature=0) and unbiased **pass@10** under stochastic sampling ($n=20$, temperature $\in \{0.2, 0.8\}$). Unless otherwise specified, “pass@10” refers to temperature=0.8; results at lower temperatures, which closely match greedy decoding, are reported on the project website [2].

To evaluate the efficacy of different iterative refinement guided methods, following [9], we adopt the following performance indicators: 1) Success Rate (SR): defined as the proportion of programs for which a valid loop invariant is successfully generated; 2) #Q: the number of SMT solver queries during inference; and 3) Time(s): the time used to infer the loop invariant. The first metric relates to effectiveness, while the latter two metrics pertain to efficiency.

3.3 Parameter Settings

Dataset Construction. We use seven state-of-the-art LLMs for both synthetic program generation (§2.3.1) and reasoning trace generation (§2.3.3): three commercial models (GPT-5-mini, GPT-4o-mini, DeepSeek-chat) and four open-source instruction-tuned models (GPTOSS-120B, Qwen3-32B, Phi-3.5, Qwen2.5-Coder-32B). Near-duplicates are removed via n -gram de-duplication. For invariant annotation (§2.3.2), we run two invariant-inference tools with a 600 s timeout and use Qwen2.5-Coder-32B as the interactive backend. For reasoning trace assessment, we combine three LLM-based evaluation criteria with equal weights to select high-quality traces.

Reasoning-oriented Fine-tuning. We implement our training pipeline on top of TRL [51] and adopt GRPO [22] for the RL stage. The RL reward combines verifier-based correctness signals from Z3 and ESBMC with a lightweight format-shaping term, following prior practice [22]. SFT+RL initializes from the SFT checkpoint, while pure RL initializes from the base model. Detailed hyperparameters, sampling configurations, and hardware setup are reported in Appendix C.3.

4 Results

4.1 Impact of Training Paradigms under Single-shot Invariant Inference

The results under the single-shot inference setting are summarized in Table 1, which presents **pass@1*** and **pass@10** for each evaluated LLM on both Clause2Inv-Bench and InvBench.

Semantic Correctness. As shown in Table 1, both SFT and SFT+RL variants consistently outperform their unfine-tuned counterparts, with SFT+RL generally achieving stronger performance than SFT.

For example, on Clause2Inv-Bench, *Qwen3-1.7B-Instruct* improves from 19.39% pass@10 to 60.90% with SFT and 61.25% with SFT+RL, corresponding to gains of over 40 percentage points. It achieves the best performance among all fine-tuned models, which we attribute to its strong latent reasoning capacity being amplified through reasoning-oriented fine-tuning. Notably, these reasoning-aware small models also surpass substantially larger open-source instruction-tuned LLMs and commercial instruction-following models—*Qwen3-1.7B-Instruct (SFT+RL)*’s 61.25% pass@10 exceeds both the larger *Qwen3-8B-Instruct* (47.41%) and *GPT-4o-mini* (44.96%). A gap remains relative to the strongest commercial reasoning LLMs (e.g., *GPT-5-mini*: 65.06% pass@1* on Clause2Inv-Bench; full results in Appendix D.2), but this likely reflects their substantially larger capacity and broader pretraining rather than limitations of reasoning-oriented fine-tuning itself.

In contrast, RL alone yields near-zero performance across both benchmarks: with only binary verification feedback, RL-trained models frequently produce degenerate outputs that fail to follow the required invariant format, preventing successful parsing. Such sparse outcome-level rewards provide limited guidance on intermediate reasoning steps (representative traces and failure examples in Appendix D.1).

Generalization to More Complex Programs. We further observe that nearly all evaluated LLMs achieve pass@1* and pass@10 below 20.00% on InvBench, revealing a substantial performance gap compared to Clause2Inv-Bench. This indicates limited generalization to structurally more complex programs. Nevertheless, improved generalization can still be observed with reasoning-oriented fine-tuning. For example, on InvBench, *Qwen3-1.7B-Instruct* achieves 13.86% pass@10 without fine-tuning, which increases to 18.54% after SFT and 18.03% with SFT+RL, corresponding to a relative improvement of over 30%. Importantly, this gain is achieved without increasing model size, suggesting that reasoning supervision helps models better transfer invariant reasoning skills to structurally more complex programs. The performance degradation on InvBench primarily reflects a distributional shift toward longer programs with richer control- and data-flow constructs (e.g., multiple loops, function calls, arrays, and pointer manipulations) that are largely absent from Clause2Inv-Bench.

4.2 Impact of Training Paradigms under Iterative-Refinement–Based Inference

Table 2 reports LaM4Inv performance with different backbone LLMs on Clause2Inv-Bench. We omit InvBench because LaM4Inv is tightly coupled to the Clause2Inv-Bench workflow and frequently fails or times out on InvBench tasks. “–” indicates #Q and Time are not applicable when no valid invariants are generated.

Refinement Effectiveness. Across all model families, reasoning-oriented fine-tuning substantially improves the SR of LaM4Inv compared to unfine-tuned backbones. Both SFT and SFT+RL variants consistently achieve large SR gains, while RL provides limited or unstable improvements. For example, *Qwen3-1.7B-Instruct* improves from 37.5% SR (vanilla) to 76.6% (SFT) and 77.8% (SFT+RL), whereas RL reaches only 30.2%. Notably, fine-tuned small models also surpass substantially larger open-source instruction-tuned LLMs within the same refinement framework. For example, *Qwen3-1.7B-Instruct (SFT+RL)* at 77.8% exceeds *Qwen3-8B-Instruct* (38.5%) and *Llama3-8B-Instruct* (25.6%), and nearly matches

Table 1: LLM Performance under single-shot Generation across Two Benchmarks. Results for commercial LLMs are provided in Appendix D.2.

Model Type	Models	Clause2Inv-Bench		invBench	
		pass@1*	pass@10	pass@1*	pass@10
OSS Instruction Models	Qwen2.5-7B-Instruct	25.77	40.69	7.62	16.56
	Qwen3-8B-Instruct	29.77	47.41	9.19	16.75
	Llama3-8B-Instruct	9.04	39.04	5.97	14.17
Fine-tuned Models	Qwen2.5-Coder-1.5B-Instruct	3.52	29.42	2.70	13.93
	Qwen2.5-Coder-1.5B-Instruct(SFT)	33.98	56.20	13.07	16.80
	Qwen2.5-Coder-1.5B-Instruct(RL)	0.00	0.25	9.81	14.75
	Qwen2.5-Coder-1.5B-Instruct(SFT+RL)	35.98	59.00	11.98	15.88
	Qwen3-1.7B-Instruct	2.69	19.39	8.19	13.86
	Qwen3-1.7B-Instruct(SFT)	35.47	60.90	10.85	18.54
	Qwen3-1.7B-Instruct(RL)	0.17	0.67	8.24	15.43
	Qwen3-1.7B-Instruct(SFT+RL)	37.15	61.25	11.46	18.03
	Llama3.2-1B-Instruct	11.22	27.60	4.31	13.07
	Llama3.2-1B-Instruct(SFT)	26.11	51.55	7.67	16.23
	Llama3.2-1B-Instruct(RL)	0.00	0.25	0.00	0.00
	Llama3.2-1B-Instruct(SFT+RL)	31.63	52.45	8.62	16.00

Table 2: Performance of LaM4Inv with Different LLMs. Results for commercial LLMs are provided in Appendix D.3.

Model Type	Models	Clause2Inv-Bench		
		SR	#Q	Time(s)
OSS-Inst. Models	Qwen2.5-7B-Instruct	64.8	8.6	54.0
	Qwen3-8B-Instruct	38.5	5.7	230.0
	Llama3-8B-Instruct	25.6	7.7	262.3
Fine-tuned Models	Qwen2.5-Coder-1.5B-Instruct	44.6	5.9	73.2
	Qwen2.5-Coder-1.5B-Instruct(SFT)	57.8	4.9	38.1
	Qwen2.5-Coder-1.5B-Instruct(RL)	40.6	5.4	12.7
	Qwen2.5-Coder-1.5B-Instruct(SFT+RL)	67.5	6.0	46.2
	Qwen3-1.7B-Instruct	37.5	6.2	267.9
	Qwen3-1.7B-Instruct(SFT)	76.6	6.7	73.5
	Qwen3-1.7B-Instruct(RL)	30.2	1.8	195.2
	Qwen3-1.7B-Instruct(SFT+RL)	77.8	7.4	73.9
	Llama3.2-1B-Instruct	20.3	6.4	55.6
	Llama3.2-1B-Instruct(SFT)	44.1	4.9	23.6
	Llama3.2-1B-Instruct(RL)	0	-	-
	Llama3.2-1B-Instruct(SFT+RL)	51.1	5.8	26.0

commercial instruction-following models such as GPT-4o-mini (within 2 points; full comparison in Appendix D.3).

These gains suggest that reasoning-oriented fine-tuning improves the quality of initial invariant candidates, making them more amenable to successful symbolic refinement. In contrast, RL relies solely on outcome-level rewards and lacks explicit reasoning supervision, resulting in poor candidate usefulness. Moreover, when equipped with reasoning-aware backbones, the iterative-refinement framework achieves substantially higher SR than the single-shot setting, indicating that symbolic feedback further amplifies the benefits of improved model reasoning.

Refinement Efficiency. Reasoning-oriented fine-tuning also improves refinement efficiency. SFT and SFT+RL models generally require fewer solver queries and less time to reach valid invariants, indicating higher-quality initial candidates that converge faster under counterexample-guided refinement. For example, Qwen2.5-Coder-1.5B-Instruct reduces average SMT queries from 5.9 to 4.9 and solving time from 73.2s to 38.1s with SFT. In contrast, RL does not consistently improve efficiency, as its low success rate often reflects failed runs rather than genuinely faster convergence.

4.3 Effectiveness of Reasoning Trace Evaluation and Selection

To evaluate our trace quality assessment criteria, we construct reasoning-annotated SFT datasets using different selection strategies and report pass@10 under the single-shot setting in Table 3: *Non-reasoning* (program-invariant pairs only), *Random*, three single-criterion variants (*Logical Consistency*, *Structural Clarity*, *Training Influence*), and *Ours* combining all three.

Reasoning supervision itself provides clear benefits—on Clause2Inv-Bench, Qwen3-1.7B-Instruct improves from 41.42 (*Non-reasoning*) to 52.42 (*Random*) pass@10, and applying structured quality assessment yields substantial further gains. The combined strategy (*Ours*) reaches 60.90, outperforming all single-criterion variants and improving by +19.48 over *Non-reasoning* and +8.48 over *Random*; consistent gains are observed on InvBench (14.17→18.54). Each individual criterion also generally outperforms *Random*, indicating that semantic and structural quality signals provide more effective supervision than random selection (e.g., Qwen2.5-Coder-1.5B-Instruct on Clause2Inv-Bench: 52.01 random vs. 56.19 *Logical Consistency*, 56.55 *Structural Clarity*). While our combined strategy achieves the most consistent gains across model-benchmark combinations, single-criterion variants occasionally surpass it on specific configurations; we provide a detailed analysis in Appendix D.4. Overall, systematically selecting traces using logical, structural, and training-aligned criteria yields substantially stronger and more reliable performance than unfiltered or single-criterion alternatives.

Table 3: Effect of Reasoning Trace Selection Strategies on SFT Performance (pass@10)

Benchmarks	Strategies	Qwen3-1.7B-Instruct	Qwen2.5-Coder-1.5B-Instruct	Llama3.2-Instruct
Clause2Inv-Bench	Non-reasoning	41.42	43.03	44.01
	Random	52.42	52.01	50.71
	Logical Consistency	60.12	56.19	50.49
	Structural Clarity	59.63	56.55	51.24
	Training Influence	58.97	56.90	50.81
	Ours	60.90	56.19	51.55
InvBench	Non-reasoning	14.17	12.96	12.85
	Random	14.72	14.59	14.69
	Logical Consistency	17.33	17.34	15.32
	Structural Clarity	16.36	16.61	16.33
	Training Influence	16.90	17.74	17.26
	Ours	18.54	16.80	16.23

4.4 Effect of Training Data Scale on Reasoning Performance

We study how training data scale affects reasoning-aware fine-tuning on Qwen3-1.7B-Instruct, the strongest fine-tuned model in earlier experiments. For SFT, we randomly sample subsets ranging from 1K to 24K examples. For SFT+RL, we apply SFT on the full 24K dataset and then continue RL training using subsets from 1K to 8K. Average pass@10 results are reported in Table 4.

On Clause2Inv-Bench, SFT performance improves steadily from 1K to 10K (54.77% to 60.61%) and saturates thereafter (60.97% at 24K). Adding RL on top of the 24K-SFT checkpoint provides further gains once the RL set is sufficiently large. With at least 5K RL examples, SFT+RL consistently surpasses SFT alone, indicating that formal-feedback-driven optimization helps refine SFT-initialized models toward verifiable invariant regions. On InvBench, gains from data scaling are more modest. SFT scaling alone shows no clear trend, while larger RL datasets yield gradual improvements, reaching 19.04% at 8K. This suggests that structural program complexity, rather than data scale, is the dominant bottleneck on InvBench.

Table 4: Pass@10 Performance of Qwen3-1.7B-Instruct under Different Training Set Sizes

Benchmark	SFT					SFT+RL			
	1K	5K	10K	15K	24K(full)	1K	3K	5K(full)	8K
Clause2Inv-bench	54.77	57.28	60.61	59.13	60.97	60.73	61.44	61.25	61.30
invBench	15.66	17.35	14.86	17.06	18.54	17.53	17.11	18.03	19.04

5 Related Work

5.1 Loop Invariant Generation Techniques

Traditional Approaches. Loop invariant inference has been extensively studied in the formal methods community using symbolic techniques such as abduction analysis [8, 15], abstract interpretation [12], Craig interpolation [28, 38], model checking and Horn clause solving [27, 50], constraint solving [48, 23], syntax-guided synthesis [19], and counterexample-guided abstraction refinement [18, 21]. Dynamic approaches such as Daikon [17] infer invariants from program executions, while modern systems like LoopInvGen [41], Eldarica [27], and SMT solvers such as CVC4/CVC5 [5] combine multiple techniques. While providing strong formal guarantees, these approaches typically depend on domain-specific abstractions or handcrafted templates, limiting their scalability to complex programs.

Learning-based Approaches. Earlier learning-based methods adopt the guess-and-check framework with various model classes, including decision trees [18, 21], support vector machines [31, 45], PAC learning [44], reinforcement learning [47], and continuous logic networks [42, 58]. For example, Code2Inv [47] uses RL to search for valid invariants but is sensitive to reward design, while CLN2INV [42] approximates logical constraints with fuzzy logic, introducing approximation errors when modeling discrete logic continuously.

More recent work explores LLMs for invariant inference, broadly falling into direct prompting [29, 10, 37, 35] and feedback-based approaches [56] that iteratively refine candidates using verification feedback. For example, Clause2Inv [9] generates and combines candidate clauses, while LaM4Inv [56] employs bounded model checking to select valid predicates from prior candidates. These approaches rely on prompting pretrained LLMs without task-specific adaptation, leaving the role of reasoning-oriented fine-tuning largely unexplored.

5.2 Reasoning-Aware Learning in Large Language Models

Explicit intermediate reasoning, often referred to as rationales or chain-of-thought (CoT), has been shown to substantially improve LLM performance across diverse tasks [32, 60, 49]. One line of work fine-tunes LLMs on datasets with step-by-step reasoning, using supervised fine-tuning or reinforcement learning to induce CoT behavior [60, 34, 22]. An alternative line leverages in-context learning by including a small number of rationale examples in the prompt [52], though such approaches generally underperform models trained with large-scale reasoning supervision and are sensitive to prompt design. Despite these advances, the impact of explicit reasoning on loop invariant inference, particularly the design and quality of reasoning supervision, has not been systematically studied.

6 Conclusion

Loop invariant inference demands deep semantic program reasoning, yet existing LLM-based approaches rarely strengthen reasoning ability beyond prompt engineering. We address this gap with a reasoning-oriented learning framework comprising a semantically validated dataset with quality-assessed reasoning traces, a trace selection strategy based on logical, structural, and training-influence criteria, and a unified fine-tuning pipeline integrating supervised reasoning signals with reinforcement learning from formal feedback. Reasoning-oriented SFT and SFT+RL substantially improve semantic correctness, generalize to more complex programs, and serve as effective backbones for symbolic refinement frameworks. In contrast, RL alone proves insufficient, underscoring the central role of structured reasoning signals. Integrating formal verification with reasoning-grounded training thus offers a promising direction for neuro-symbolic program analysis.

References

- [1] Collection of verification tasks (moved, please follow the link), 2025-3.
- [2] Loop invariant infer only once, 2025-3.
- [3] A syntax-guided synthesis problem (sygus), 2025-3.
- [4] tree sitter for c programming, 2025-3.
- [5] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. cvc5: A versatile and industrial-strength smt solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 415–442. Springer, 2022.
- [6] Marc Brockschmidt, Yuxin Chen, Byron Cook, Pushmeet Kohli, and Daniel Tarlow. Learning to decipher the heap for program verification. In *Workshop on Constructive Machine Learning at the International Conference on Machine Learning (CMLICML)*, volume 21, 2015.
- [7] Marc Brockschmidt, Yuxin Chen, Pushmeet Kohli, Siddharth Krishna, and Daniel Tarlow. Learning shape analysis. In *International Static Analysis Symposium*, pages 66–87. Springer, 2017.
- [8] Cristiano Calcagno, Dino Distefano, and Viktor Vafeiadis. Bi-abductive resource invariant synthesis. In *Asian Symposium on Programming Languages and Systems*, pages 259–274. Springer, 2009.
- [9] Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025.
- [10] Saikat Chakraborty, Shuvendu Lahiri, Sarah Fakhoury, Akash Lal, Madanlal Musuvathi, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking llm-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9164–9175, 2023.
- [11] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [12] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252. ACM, 1977.
- [13] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.
- [14] Edsger W Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975.
- [15] Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. *Acm Sigplan Notices*, 48(10):443–456, 2013.
- [16] Yangruibo Ding, Jinjun Peng, Marcus Min, Gail Kaiser, Junfeng Yang, and Baishakhi Ray. Semcoder: Training code language models with comprehensive semantics reasoning. *Advances in Neural Information Processing Systems*, 37:60275–60308, 2024.
- [17] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1-3):35–45, 2007.
- [18] P Ezudheen, Daniel Neider, Deepak D’Souza, Pranav Garg, and P Madhusudan. Horn-ice learning for synthesizing invariants and contracts. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–25, 2018.

- [19] Grigory Fedyukovich and Rastislav Bodík. Accelerating syntax-guided invariant synthesis. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 251–269. Springer, 2018.
- [20] Juan P Galeotti, Carlo A Furia, Eva May, Gordon Fraser, and Andreas Zeller. Inferring loop invariants by mutation, dynamic analysis, and static checking. *IEEE transactions on software engineering*, 41(10):1019–1037, 2015.
- [21] Pranav Garg, Daniel Neider, Parthasarathy Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. *ACM Sigplan Notices*, 51(1):499–512, 2016.
- [22] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [23] Ashutosh Gupta, Rupak Majumdar, and Andrey Rybalchenko. From tests to proofs. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 262–276. Springer, 2009.
- [24] Matthias Heizmann, Jochen Hoenicke, and Andreas Podelski. Software model checking for people who love automata. In *International Conference on Computer Aided Verification*, pages 36–52. Springer, 2013.
- [25] Vincent J Hellendoorn, Premkumar T Devanbu, Oleksandr Polozov, and Mark Marron. Are my invariants valid? a learning approach. *arXiv preprint arXiv:1903.06089*, 2019.
- [26] Charles Antony Richard Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [27] Hossein Hojjat and Philipp Rümmer. The eldarica horn solver. In *2018 Formal Methods in Computer Aided Design (FMCAD)*, pages 1–7. IEEE, 2018.
- [28] Ranjit Jhala and Kenneth L McMillan. A practical and complete approach to predicate refinement. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 459–473, 459–473, 2006. Springer, 459–473.
- [29] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- [30] Tyler Lanham, A. Chen, A. Radhakrishnan, B. Steiner, G. Sutton, C. von Beusekom, N. Chaudhuri, and D. Dohan. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint*, 2023.
- [31] Jiaying Li, Jun Sun, Li Li, Quang Loc Le, and Shang-Wei Lin. Automatic loop-invariant generation and refinement through selective sampling. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 782–792. IEEE, 2017.
- [32] Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. Making language models better reasoners with step-aware verifier. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5315–5333, 2023.
- [33] Hunter Lightman et al. Let’s verify step by step. *arXiv preprint*, 2023.
- [34] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint*, 2023.
- [35] Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: a benchmark of programs with memory manipulation. *Advances in Neural Information Processing Systems*, 37:129120–129145, 2024.

- [36] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36:21558–21572, 2023.
- [37] Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *Science of Computer Programming*, page 103387, 2025.
- [38] Kenneth L McMillan. Lazy annotation for program testing and verification. In *International Conference on Computer Aided Verification*, pages 104–118. Springer, 2010.
- [39] Rafael Menezes, Mohannad Aldughaim, Bruno Farias, Xianzhiyu Li, Edoardo Manino, Fedor Shmarov, Kunjian Song, Franz Brauße, Mikhail R. Gadelha, Norbert Tihanyi, Konstantin Korovin, and Lucas C. Cordeiro. ESBMC 7.4: Harnessing the Power of Intervals. In *30th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS’24)*, volume 14572 of *Lecture Notes in Computer Science*, page 376–380. Springer, 2024.
- [40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- [41] Saswat Padhi, Rahul Sharma, and Todd Millstein. Loopinvgen: A loop invariant generator based on precondition inference. *arXiv preprint arXiv:1707.02029*, 2017.
- [42] Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. Cln2inv: learning loop invariants with continuous logic networks. *arXiv preprint arXiv:1909.11542*, 2019.
- [43] Rahul Sharma and Alex Aiken. From invariant checking to invariant inference using randomized search. *Formal Methods in System Design*, 48(3):235–256, 2016.
- [44] Rahul Sharma, Saurabh Gupta, Bharath Hariharan, Alex Aiken, and Aditya V Nori. Verification as learning geometric concepts. In *International Static Analysis Symposium*, pages 388–411. Springer, 2013.
- [45] Rahul Sharma, Aditya V Nori, and Alex Aiken. Interpolants as classifiers. In *International Conference on Computer Aided Verification*, pages 71–87. Springer, 2012.
- [46] Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. *Advances in Neural Information Processing Systems*, 31, 2018.
- [47] Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2inv: A deep learning framework for program verification. In *International Conference on Computer Aided Verification*, pages 151–164. Springer, 2020.
- [48] Constraint Solving. Linear invariant generation using non-linear. *Computer-aided Verification: Proceedings*, page 420, 2003.
- [49] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.
- [50] Hari Govind Vadiramana Krishnan, YuTing Chen, Sharon Shoham, and Arie Gurfinkel. Global guidance for local generalization in model checking. *Formal Methods in System Design*, 63(1):81–109, 2024.
- [51] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020.
- [52] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint*, 2022.

- [53] Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. Invbench: Can llms accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.
- [54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [55] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro Von Werra, Arjun Guha, and Lingming Zhang. Selfcodealign: Self-alignment for code generation. *Advances in Neural Information Processing Systems*, 37:62787–62874, 2024.
- [56] Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Llm meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 406–417, 2024.
- [57] Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. *arXiv preprint arXiv:2310.04870*, 2023.
- [58] Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 106–120, 2020.
- [59] Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through smt solving enhanced reinforcement learning. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 175–187, 2023.
- [60] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488, 2022.
- [61] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.

Part I

Appendix

Contents

A	Reproducibility Materials	14
B	Prompt Templates and Examples	14
B.1	Prompt for Generating Loop Programs	14
B.2	Translation-induced False Positives	15
B.3	Reasoning Trace Generation Prompts	15
B.4	Reasoning Trace Quality Criteria	15
B.5	SFT Prompt Template	16
C	Experimental Setup Details	16
C.1	Pass@k Computation	16
C.2	Evaluation Benchmarks	16
C.3	Parameters Details	17
D	Experimental Result Details	17
D.1	Examples of Reasoning Traces Generated by Different Models	17
D.2	Complete Performance Results for Single-shot Generation	18
D.3	Complete Performance Results for LaM4Inv	19
D.4	Detailed Trace Selection Ablation	19
E	Limitation	20
F	Impact of Our Work	20

A Reproducibility Materials

A permanent public repository has not yet been linked in this preprint. The following appendices document prompt templates, hyperparameters, and additional results.

B Prompt Templates and Examples

B.1 Prompt for Generating Loop Programs

Figure 2 presents the full prompt template used in our program synthesis step (Section 2.3.1). The template instantiates the three components introduced in the main text: a task instruction, a generation specification, and a seed code snippet, with concrete formatting and placeholder fields. The generation specification enforces two categories of constraints. Structural constraints require each generated program to (i) contain a main function, (ii) preserve the structure of the seed loop, (iii) use nondeterministic initialization for input variables, and (iv) apply assume statements to constrain variable ranges, ensuring compatibility with downstream SMT-based verification. Diversity constraints vary loop difficulty (easy, medium, hard) and target functionality (e.g., pattern matching, numerical computation, array traversal, index tracking), promoting coverage across program patterns.

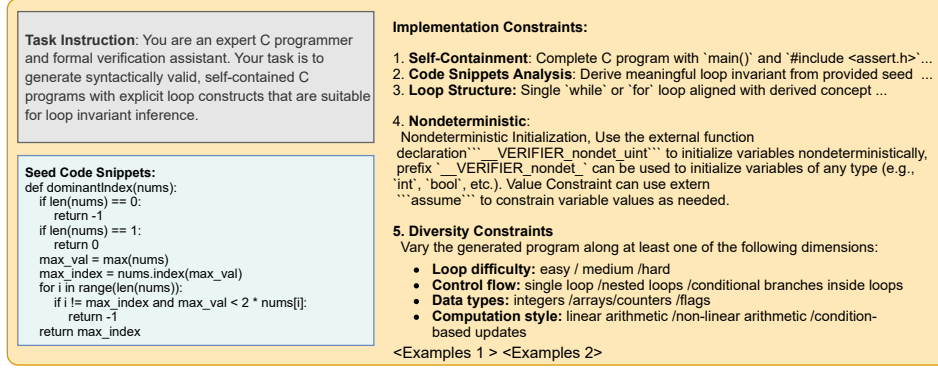


Figure 2: Example of prompt for Generating Loop Programs.

B.2 Translation-induced False Positives

Figure 3 shows an example of such translation-induced false positives: LaM4Inv’s translator incorrectly converts the C-level candidate invariant `assert(0 <= numbers[j] && numbers[j] <= 1000)` into the SMT assertion (`assert 1000`).

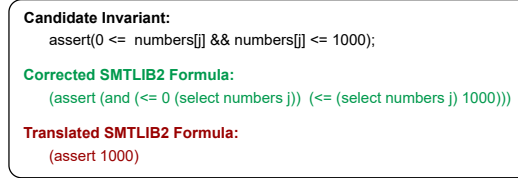


Figure 3: Incorrect Translation of a C-level Candidate Invariant into SMT Formula

B.3 Reasoning Trace Generation Prompts

This appendix provides the detailed prompting strategy used to generate reasoning traces for verified program-invariant pairs (Section 2.3.3). Each prompt instructs a strong LLM to produce a reasoning trace structured around four components: *Loop and State Abstraction*, *Behavioral Pattern Extraction*, *Constraint Generalization*, and *Invariant Synthesis*. We describe each component below.

- **Loop and State Abstraction.** The trace identifies loop variables, control conditions, and relevant state components (e.g., counters, array indices, accumulators), and summarizes how these variables evolve across iterations. This step abstracts the program into a symbolic transition view that highlights stable relationships and monotonic trends.
- **Behavioral Pattern Extraction.** The trace analyzes the loop body to infer recurring update patterns, such as monotonic changes, bounded ranges, conservation relationships, or index–value coupling (e.g., “i increases by 1 while sum accumulates a[i]”). These patterns serve as candidate semantic constraints.
- **Constraint Generalization.** The trace generalizes the extracted patterns into symbolic constraints over variables, such as bounds, relational inequalities, or structural properties, and explains why these constraints are expected to hold across all iterations.
- **Invariant Synthesis.** Finally, the trace combines the derived constraints into a coherent invariant formula and explains how this formula captures the stable semantic properties of the loop.

B.4 Reasoning Trace Quality Criteria

This appendix provides the detailed specifications of the three quality criteria used to evaluate candidate reasoning traces (Section 2.3.3). Each candidate trace is independently scored on each criterion by an LLM-based evaluator on a scale from 1 (worst) to 5 (best), yielding a quality vector $\mathbf{q} = (q_{lc}, q_{sc}, q_{ti}) \in [1, 5]^3$.

1. **Logical Consistency** (q_{lc}). This criterion evaluates whether the reasoning trace and the target invariant are logically consistent and mutually supportive under the semantics of the program. The invariant should be derivable from the trace, and each step must be grounded in the concrete behavior of the program, its control flow, variable updates, or previously established facts. This criterion penalizes post hoc explanations, circular reasoning, and abstract statements that are not semantically tied to the program.
2. **Structural Clarity** (q_{sc}). This criterion measures how clearly the trace is organized as a derivation process for the invariant. A high-quality trace should present a coherent progression from state abstraction and behavioral pattern extraction to constraint generalization and invariant synthesis. Each step should contribute meaningfully to the final invariant, avoiding redundant, tangential, or filler content.
3. **Training Influence** (q_{ti}). This criterion estimates the potential contribution of a trace to effective model training. Higher scores are assigned to traces associated with rare or structurally complex invariants (e.g., non-linear arithmetic, modular relationships, or bit-level properties), based on structure-level rarity estimates. This encourages the selection of traces that promote generalization and mitigates overfitting to frequent linear or template-like patterns.

B.5 SFT Prompt Template

Figure 4 shows the full instruction-following template used during supervised fine-tuning (Section 2.4). The system message instructs the model to produce an internal reasoning monologue followed by a final answer, both enclosed in dedicated tags so that training and inference can consistently extract the invariant.

```
[System:] A conversation between User and Assistant. The assistant first
think about the reasoning process as an internal monologue and then provide
the user with the answer. Respond in the following format, enclosed within
<think>...</think> and <answer>...</answer> tags, respectively, i.e.,
<think>reasoning process here</think> <answer>answer here</answer>.
[User:] /* Program code here */
[Assistant:] <think>T</think> <answer>I</answer>
```

Figure 4: Prompt template used for SFT training. T denotes the curated reasoning trace and I the verified loop invariant.

C Experimental Setup Details

C.1 Pass@k Computation

We adopt the unbiased pass@k estimator from [11]. For each program, we sample $n \geq k$ candidate invariants from the model, among which $c \leq n$ are deemed correct by passing the formal verifier. The unbiased pass@k is computed as:

$$\text{pass}@k := \mathbb{E}_{\text{Program}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]. \quad (3)$$

This estimator avoids the high variance of the naive empirical fraction $1 - (1 - c/n)^k$, particularly when n is small. In our experiments, we use $n = 20$ and report pass@10 by default.

C.2 Evaluation Benchmarks

We use two widely adopted benchmarks to evaluate the semantic correctness and generalization of learned invariant reasoning.

Clause2Inv-Bench [9] is a curated benchmark that aggregates problems from multiple sources, including Code2Inv [47], the 2019 SyGuS competition [3], the 2024 SV-COMP benchmarks [59],

and LIPuS [1]. It consists of 316 linear and 50 non-linear loop invariant synthesis tasks and has been widely used in prior work to evaluate invariant inference techniques.

InvBench [53] is constructed from a pool of SV-COMP [1] verification tasks that require loop invariants, and is designed to assess generalization across diverse program structures and verification conditions. It retains 226 instances that can be solved by UAutomizer [24] within a 600-second timeout.

Combining these benchmarks, we obtain a comprehensive evaluation set of 592 tasks for our experimental study.

C.3 Parameters Details

Parameters in Dataset Construction. We conduct both *Synthetic Program Generation* (§2.3.1) and *Reasoning Trace Generation* (§2.3.3) using seven state-of-the-art LLMs: three commercial models (GPT-5-mini, GPT-4o-mini, and DeepSeek-chat) and four open-source instruction-tuned models (GPTOSS-120B, Qwen3-32B, Phi-3.5, and Qwen2.5-Coder-32B). During program generation, we sample each seed one time for commercial models and 16 times for open-source models; we use the same sampling strategy for trace generation, except that open-source models are sampled only 4 times per instance. We remove near-duplicates using 8-gram de-duplication.

In the *Invariant Annotation* stage (§2.3.2), we run two invariant-inference tools with a 600s timeout and use Qwen2.5-Coder-32B as the interactive backend. For *Reasoning Trace Assessment*, we apply three evaluation criteria by prompting the same set of models used above, excluding DeepSeek-chat. The three criteria are combined with equal weights (1.0 each) to select high-quality reasoning traces.

Parameters in Reasoning-oriented Fine-tuning. For the fine-tuning of the three models (SFT, RL, and SFT+RL), we develop our training framework based on TRL [51]. For SFT, we train them in bfloat16 with FlashAttention-2 enabled, launching accelerate with FSDP for memory efficiency. We use a maximum sequence length of 4096 with packed examples, a per-device batch size of 16, a learning rate of 5×10^{-5} under a cosine schedule with 5% warmup, and train for 16 epochs. For RL with formal feedback, we launch accelerate with DDP (rather than FSDP) due to the higher GPU memory overhead of FSDP in RL. We initialize from the SFT checkpoint (SFT+RL) or from the base model (RL), and apply GRPO [22] with a learning rate of 2×10^{-5} , cosine scheduling, and 10% warmup for 5 epochs, using a per-device batch size of 16. During GRPO’s online generation, we sample 16 completions per prompt, the RL reward is a weighted sum of verifier-based correctness signals from Z3 and ESBMC (weight 1.0 each) and lightweight any non-functionality with weight 0.1, such as format-shaping, CONVERT etc., following prior practice [22]. All training and evaluation are conducted on a single node with 4×H200-SXM (140,GB) and an AMD EPYC 9554P CPU (64 cores / 128 threads, 3.54 GHz).

D Experimental Result Details

D.1 Examples of Reasoning Traces Generated by Different Models

Figure 5 shows the examples of reasoning traces generated by different models. We can see that reasoning traces generated by Qwen3-1.7B-Instruct (SFT+RL) often exhibit intermediate self-refinement, in which earlier incorrect assumptions are revised in later steps of the reasoning process. This pattern indicates that the base model possesses a latent capacity for structured reasoning, which can be further amplified through reasoning-oriented fine-tuning. RL-trained models frequently produce degenerate outputs with excessive filler tokens and fail to follow the required invariant format, preventing successful parsing and verification. This behavior is consistent with RL’s training signal, which relies solely on binary verification feedback. Such sparse, outcome-level rewards provide limited guidance on intermediate reasoning steps, making it difficult for the policy to discover constructive update directions. As a result, the training process often fails to converge in the absence of more informative supervision.

Question (a)	Reasoning Model's Response
[System] You are given a C program with a loop. Write an assert(...) that encodes a loop invariant (same style as assert(x <= 1)), i.e., a boolean condition that should hold at the loop head across iterations and be preserved by the loop body. [User] ... <pre> int main() { int x=_VERIFIER_nondet_int(); int y=_VERIFIER_nondet_int(); assume_abort_if_not(0<=x && x<=1 && 0<=y && y<=1 && x>=1 && y>=1); long long a=x,b=y,p=1,q=0,r=0,s=1; // Invariant will be injected here to verify. while(b){ long long c=a,k=0; while(c>=b){ long long d=1,v=b; while(c>=2*v){ __VERIFIER_assert(a==k*b+c); d*=2; v*=2; } c-=v; k+=d; } } return 0; } </pre>	Qwen3-1.7B: (b) <think> Okay, let's tackle this problem. The task...First, I need to understand ... extended Euclidean algorithm. The main loop is a while loop that ... Looking at the code, the variables a and b are The variables p, q, r, s ... the condition is while (1) and checks if b is not zero. The inner loop ... The assertions inside the inner loop ensure that a == k*b + c. Wait, the outer loop's condition is checking if b is not zero. So, the invariant must hold before the loop body is executed. The loop body is executed only when b is not zero. But the loop is infinite, so the invariant must hold for all iterations. The key is to find Hmm, the exact invariant is tricky. The code has a lot of assertions, and the loop invariant must be derived from those. Aha! Considering the code's inner loop, the assertion a == k*b + c is part of the loop's invariant. So, the loop invariant might include that assertion. Given the loop invariant for the outer loop is likely: Final Answer: assert(a == k * b + c); </think>
Two Failure Models	Error message from ESBMC
Qwen2.5-Coder-1.5B(RL): (c) <pre> /**Computethe...inyour<answer>;output...<answer>/ ***Example:*****/[omit200]****/ ***/***/ </pre> Qwen3-1.7B(RL): (d) <pre> <think> assert(b < a + b); </think> </pre>	The invariant include "c" will triggle parsing error. Parsing /tmp/tmpxkouuacv/check.c /tmp/tmpxkouuacv/check.c:40:41: error: use of undeclared identifier 'c' __ESBMC_loop_invariant(a == k * b + c); ERROR: PARSING ERROR...

Figure 5: Examples of Reasoning Traces Generated by Different Models

D.2 Complete Performance Results for Single-shot Generation

Table 5 presents complete performance results across all evaluated models, including commercial reasoning LLMs. For commercial reasoning models (GPT-5-mini, GPT-o4-mini, Deepseek-reasoner), we report only pass@1* due to the substantial computational and monetary cost of multi-sample evaluation. With only 1.7B parameters, Qwen3-1.7B-Instruct (SFT+RL) achieves 37.15% pass@1* on Clause2Inv-Bench, surpassing commercial instruction-following models (e.g., GPT-4o-mini: 23.94%) but remaining below commercial reasoning LLMs (49.51–65.06%). The remaining gap likely reflects the substantially larger parameter count, broader pretraining, and dedicated reasoning-oriented post-training of commercial reasoning systems, rather than limitations of reasoning-oriented fine-tuning itself.

Table 5: LLM Performance under Single-shot Generation across Two Benchmarks.

Model Type	Models	Clause2Inv – Bench		invBench	
		pass@1*	pass@10	pass@1*	pass@10
Commercial Reasoning	GPT-5-mini	65.06	-	18.00	-
	GPT-o4-mini	52.34	-	16.72	-
	Deepseek-reasoner	49.51	-	11.37	-
Commercial Instruction	GPT-4o-mini	23.94	44.96	11.37	17.63
	Deepseek-chat	51.04	78.85	9.76	17.12
OSS Instruction Models	Qwen2.5-7B-Instruct	25.77	40.69	7.62	16.56
	Qwen3-8B-Instruct	29.77	47.41	9.19	16.75
	Llama3-8B-Instruct	9.04	39.04	5.97	14.17
Fine-tuned Models	Qwen2.5-Coder-1.5B-Instruct	3.52	29.42	2.70	13.93
	Qwen2.5-Coder-1.5B-Instruct(SFT)	33.98	56.20	13.07	16.80
	Qwen2.5-Coder-1.5B-Instruct(RL)	0.00	0.25	9.81	14.75
	Qwen2.5-Coder-1.5B-Instruct(SFT+RL)	35.98	59.00	11.98	15.88
	Qwen3-1.7B-Instruct	2.69	19.39	8.19	13.86
	Qwen3-1.7B-Instruct(SFT)	35.47	60.90	10.85	18.54
	Qwen3-1.7B-Instruct(RL)	0.17	0.67	8.24	15.43
	Qwen3-1.7B-Instruct(SFT+RL)	37.15	61.25	11.46	18.03
	Llama3.2-1B-Instruct	11.22	27.60	4.31	13.07
	Llama3.2-1B-Instruct(SFT)	26.11	51.55	7.67	16.23
	Llama3.2-1B-Instruct(RL)	0.00	0.25	0.00	0.00
	Llama3.2-1B-Instruct(SFT+RL)	31.63	52.45	8.62	16.00

D.3 Complete Performance Results for LaM4Inv

Table 6 presents the complete performance results for LaM4Inv on Clause2Inv-Bench across all evaluated backbone LLMs, including 2 commercial instruction-following models, 3 open-source instruction-tuned models, and our 12 fine-tuned models. We report success rate (SR), the average number of SMT queries to reach a valid invariant ($\#Q$), and the average solving time (Time, in seconds); “–” indicates cases where no valid invariants are generated, making $\#Q$ and Time undefined. With only 1.7B parameters, Qwen3-1.7B-Instruct (SFT+RL) achieves 77.8% SR, surpassing all open-source instruction-tuned baselines (Qwen2.5-7B-Instruct: 64.8%; Qwen3-8B-Instruct: 38.5%; Llama3-8B-Instruct: 25.6%) and nearly matching commercial instruction-following models (within 1.5 points of GPT-4o-mini’s 79.3%, and 4.2 points below Deepseek-Chat’s 82.0%). The remaining gap likely reflects the substantially larger parameter count and broader pretraining of commercial models, rather than limitations of reasoning-oriented fine-tuning itself. Beyond invariant quality, our fine-tuned model also achieves substantial efficiency gains, requiring only 73.9 seconds on average compared to 230.0s for Qwen3-8B-Instruct and 262.3s for Llama3-8B-Instruct, indicating that reasoning-oriented fine-tuning enables compact backbones to provide higher-quality and more efficient invariant candidates within the iterative refinement framework.

Table 6: Performance of LaM4Inv with Different Backbone LLMs.

Model Type	Models	Clause2Inv-Bench		
		SR	#Q	Time(s)
Commercial Models	GPT-4o-mini	79.3	12.0	152.7
	Deepseek-chat	82.0	2.3	187.6
OSS-Inst. Models	Qwen2.5-7B-Instruct	64.8	8.6	54.0
	Qwen3-8B-Instruct	38.5	5.7	230.0
	Llama3-8B-Instruct	25.6	7.7	262.3
Fine-tuned Models	Qwen2.5-Coder-1.5B-Instruct	44.6	5.9	73.2
	Qwen2.5-Coder-1.5B-Instruct(SFT)	57.8	4.9	38.1
	Qwen2.5-Coder-1.5B-Instruct(RL)	40.6	5.4	12.7
	Qwen2.5-Coder-1.5B-Instruct(SFT+RL)	67.5	6.0	46.2
	Qwen3-1.7B-Instruct	37.5	6.2	267.9
	Qwen3-1.7B-Instruct(SFT)	76.6	6.7	73.5
	Qwen3-1.7B-Instruct(RL)	30.2	1.8	195.2
	Qwen3-1.7B-Instruct(SFT+RL)	77.8	7.4	73.9
	Llama3.2-1B-Instruct	20.3	6.4	55.6
	Llama3.2-1B-Instruct(SFT)	44.1	4.9	23.6
	Llama3.2-1B-Instruct(RL)	0	-	-
	Llama3.2-1B-Instruct(SFT+RL)	51.1	5.8	26.0

D.4 Detailed Trace Selection Ablation

We provide additional analysis on model- and benchmark-dependent behaviors observed in our trace selection ablation (Table 3).

Model-Dependent Variations across Single Criteria. While our combined strategy (*Ours*) achieves the strongest or tied-strongest performance on most model–benchmark combinations, individual single-criterion variants occasionally surpass it. For Qwen2.5-Coder-1.5B-Instruct, *Training Influence* slightly outperforms *Ours* on both Clause2Inv-Bench (56.90 vs. 56.19) and InvBench (17.74 vs. 16.80). We hypothesize that the combined assessment may filter out traces that align particularly well with this model’s coding-oriented inductive biases. For Llama3.2-1B-Instruct, *Logical Consistency* alone performs slightly worse than *Random* on Clause2Inv-Bench (50.49 vs. 50.71), although it remains competitive on InvBench (15.32 vs. 14.69). This suggests that logical correctness without structural alignment may provide inconsistent supervision signal for smaller, less capable backbones.

Benchmark-Dependent Effects. The relative ranking of selection strategies is not entirely stable across benchmarks. For example, on Qwen3-1.7B-Instruct, *Logical Consistency* (60.12) approaches *Ours* (60.90) on Clause2Inv-Bench but achieves the second-best result on InvBench (17.33 vs. *Ours* 18.54). This may reflect that different benchmarks emphasize different aspects of reasoning quality—*Clause2Inv-Bench* tasks, with simpler control flow, may benefit most from logically tight traces, while the more complex *InvBench* programs reward the broader trace diversity captured by the combined strategy.

Effect of Backbone Capacity. Across all three backbones, the absolute gain from reasoning supervision (relative to *Non-reasoning*) is largest for Qwen3-1.7B-Instruct (+19.48 on Clause2Inv-Bench, +4.37 on InvBench under *Ours*) and smallest for Llama3.2-1B-Instruct (+7.54 and +3.38 respectively). This is consistent with our observation in RQ1 that backbones with stronger latent reasoning capacity benefit more from reasoning-oriented supervision; weaker backbones see meaningful but more modest gains.

Discussion. These observations indicate that while our combined trace selection strategy provides the most consistent and robust gains across model–benchmark combinations, individual model architectures and benchmark characteristics may favor different criterion weightings. A more principled, model-adaptive weighting scheme—potentially learned per backbone—is a promising direction for future work.

E Limitation

Internal Validity. The quality of reasoning traces is assessed using LLM-based judges, which provide heuristic rather than formal evaluations. Although we employ multiple complementary criteria and strict filtering on critical dimensions such as logical soundness and program grounding, the assessment may still introduce noise or bias. Our verification pipeline relies on SMT solvers and bounded model checking with a bounded unwinding $k < 10$ for tractable online RL training, which may introduce false positives by accepting invariants that hold only under bounded execution. We mitigate this through semantic consistency checks and diverse GRPO rollouts, but residual tool-specific effects may remain.

External Validity Our fine-tuning experiments focus on relatively small LLMs due to computational resource constraints, which may limit the direct generalizability of training results to very large commercial models. However, we additionally evaluate a range of larger pretrained and commercial models during inference, and observe consistent trends regarding the impact of explicit reasoning. Moreover, the training data used in this study is synthetically constructed, enabling controlled invariant annotation and reasoning supervision but potentially omitting certain real-world complexities such as extensive library interactions or environment-dependent behavior. Consequently, performance on real-world codebases may differ from the results reported here.

F Impact of Our Work

Our work on automated loop invariant inference can support software reliability and formal verification, with potential positive impacts in safety-critical domains such as aerospace, medical devices, and autonomous systems. By enabling compact open-source LLMs to perform invariant reasoning, our approach also lowers the resource barrier for neural-symbolic research. Potential negative impacts include over-reliance on automatically generated invariants without independent verification; we mitigate this by integrating formal verification (SMT solvers and bounded model checking) directly into our reward signal, ensuring that retained outputs are verifier-validated rather than purely model-generated.